

Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V) Dot .Net Core
and Progressive Web Application Practical

Practical – V

Roll No.:	Name:
Class:	Batch:
Date of Assignment:	Date/Time of Submission:

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Start.

Load WebForm1.

Check whether the page is loaded for the first time.

If No (PostBack):

- Read the value stored in ViewState["count"].
- Increment the value by 1.
- Display the incremented value in Label2.

If Yes (First Load):

- Store 1 in ViewState["count"].

When Button1 is clicked:

- Read ViewState["count"].
- Display its value in Label1.

When Button2 is clicked:

- Create a cookie named "name".

- Store the text entered in TextBox1 into the cookie.
- Add the cookie to the response.
- Redirect the user to WebForm2.aspx.

Stop.

Start.

Load WebForm2.

Check whether the cookie "name" exists.

If the cookie exists:

- Read the value stored in the cookie.
- Display "Welcome" along with the cookie value.

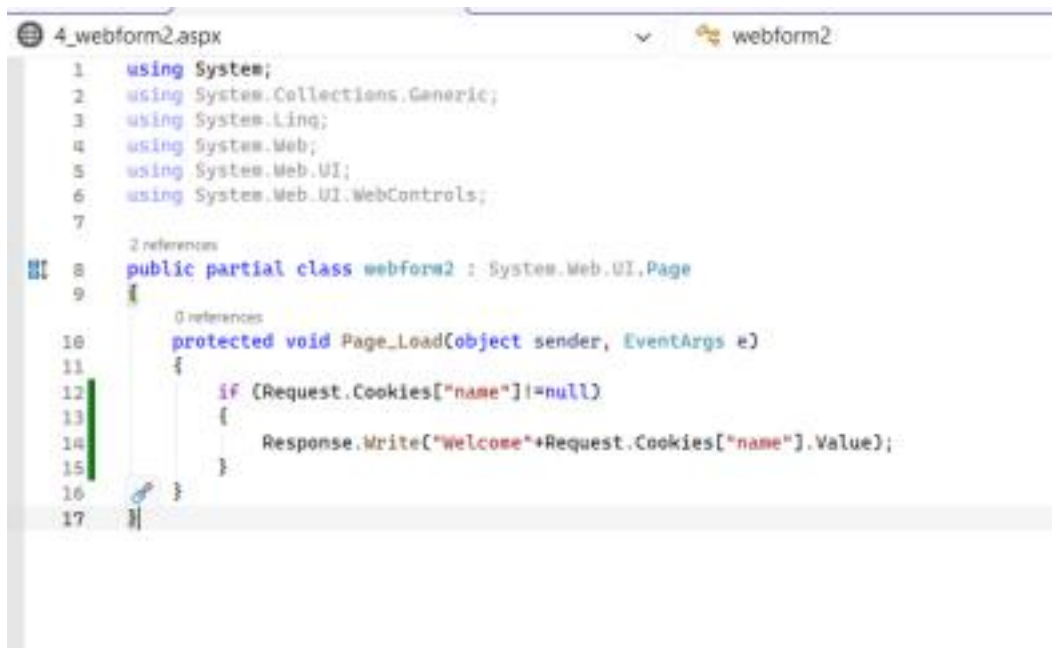
If the cookie does not exist:

- Do nothing.

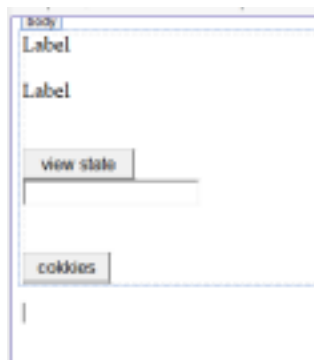
Stop.

Code:

```
webform1.aspx webform_1
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.UI;
6 using System.Web.UI.WebControls;
7
8 // references
9 public partial class webform_1 : System.Web.UI.Page
10 {
11     // references
12     protected void Page_Load(object sender, EventArgs e)
13     {
14         if (IsPostBack)
15         {
16             int ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
17             Label1.Text = "view state = " + ViewStateVal;
18         }
19         else
20         {
21             ViewState["count"] = 1;
22         }
23     }
24
25     // references
26     protected void Button1_Click(object sender, EventArgs e)
27     {
28         Label1.Text = ViewState["count"].ToString();
29     }
30
31     // references
32     protected void Button2_Click(object sender, EventArgs e)
33     {
34         HttpCookie h = new HttpCookie("name");
35         h.Value = TextBox1.Text;
36         Response.Cookies.Add(h);
37         Response.Redirect("WebForm2.aspx");
38     }
39 }
```

Code:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.UI;
6 using System.Web.UI.WebControls;
7
8 public partial class webform2 : System.Web.UI.Page
9 {
10     protected void Page_Load(object sender, EventArgs e)
11     {
12         if (Request.Cookies["name"] != null)
13         {
14             Response.Write("Welcome" + Request.Cookies["name"].Value);
15         }
16     }
17 }
```

Output:

The screenshot shows a web application interface. At the top, there is a label 'Label'. Below it, another label 'Label' is visible. A button labeled 'view state' is positioned above a text input field that contains the text 'som mishra'. Below the input field is a button labeled 'cookies'. At the bottom of the visible area, there is a message 'Welcome som mishra' followed by another 'Label'.

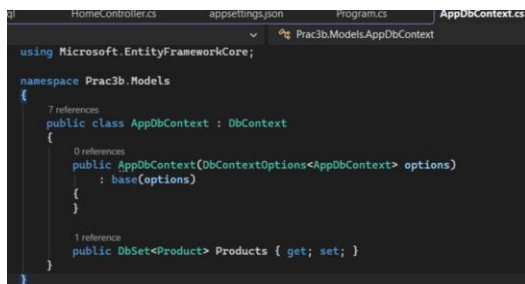
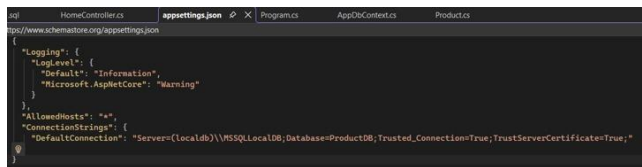
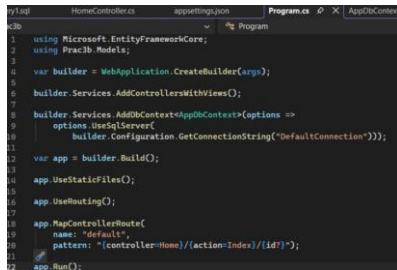
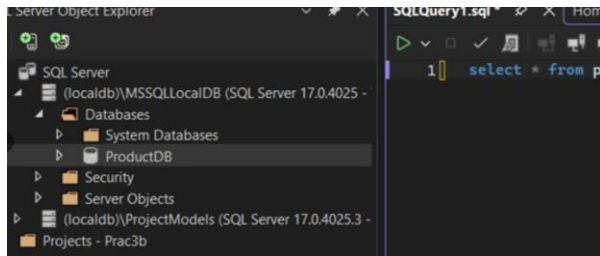
b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

1. Start.
2. Create a database named ProductDB.
3. Create the Product model with Id, Name, and Price.
4. Create ApplicationDbContext and add the Products table (DbSet).
5. Configure the SQL Server connection string in appsettings.json.
6. Register ApplicationDbContext in Program.cs.
7. Create HomeController and inject ApplicationDbContext.
8. Create a new product object and assign values (e.g., Laptop, 50000).
9. Add the product to the database using Add() and save it using SaveChanges().
10. Install Entity Framework Core packages.
11. Create and apply migrations using: o Add-Migration InitialCreate o Update-Database
12. Run the application.
13. Open Home/Index to insert the product.
14. Verify the inserted record in SQL Server using SELECT * FROM Products.
15. Stop.

Code:

SOM T023



Output:

Product Inserted Successfully

Id	Name	Price
1	Laptop Som	50000.00

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm:

1. Start.
2. Open Visual Studio and create an ASP.NET Core MVC project.
3. Create the Employee model with required properties.
4. Create the ApplicationDbContext class.
5. Add DbSet to the context.
6. Configure the connection string in appsettings.json.
7. Register ApplicationDbContext in Program.cs.
8. Install Entity Framework Core packages.
9. Create and apply database migrations.
10. Create the EmployeesController.
11. Implement the Create operation to add employee records.
12. Implement the Read operation to display employee records.
13. Implement the Edit operation to update employee records.
14. Implement the Delete operation to remove employee records.
15. Run the application.
16. Test Create, Read, Edit, and Delete operations.
17. Verify the output.
18. Stop.

Code:-

EmployeesController.cs:

SOM T023

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
using EmployeeCRUD.Data;

1 reference
public class EmployeesController : Controller
{
    private readonly AppDbContext _context;

    0 references
    public EmployeesController(AppDbContext context)
    {
        _context = context;
    }

    // GET: EMPLOYEES
    3 references
    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }

    // GET: EMPLOYEES/Details/5
    0 references
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {

```

Employee.cs:

```
namespace EmployeeCRUD.Models
{
    13 references
    public class Employee
    {
        11 references
        public int ID { get; set; }
        12 references
        public required string Name { get; set; }
        12 references
        public required string Department { get; set; }
        12 references
        public double Salary { get; set; }
    }
}
```

AppDbContext.cs:

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
{
    7 references
    public class AppDbContext:DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options){}
        7 references
        public DbSet<Employee> Employees { get; set; }
    }
}
```

Appsetting.json:

```
{
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MyConnection": "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext": "Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

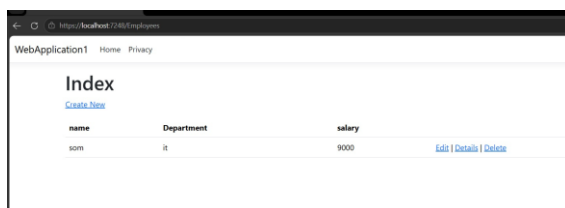
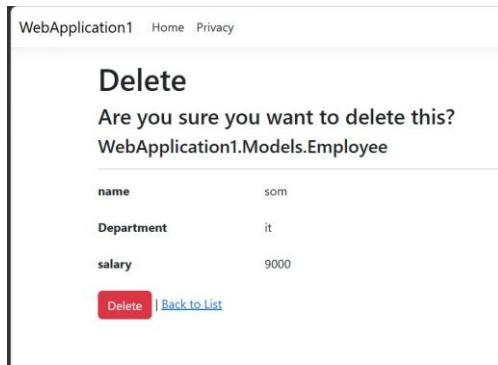
Program.cs:

```

EmployeeCRUD
1 using Microsoft.EntityFrameworkCore;
2 using EmployeeCRUD.Data;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 // Add services to the container.
7 builder.Services.AddControllersWithViews();
8
9 builder.Services.AddDbContext<AppDbContext>(<options> => options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
10
11 var app = builder.Build();
12
13 // Configure the HTTP request pipeline.
14 if (!app.Environment.IsDevelopment())
15 {
16     app.UseExceptionHandler("/Home/Error");
17     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
18     app.UseHsts();
19 }

```

Output:-



d) Authentication and Authorization in ASP.NET Core.

Algorithm:

1. Create an ASP.NET Core MVC project.
2. Enable Cookie Authentication in Program.cs.
3. Create Login action in HomeController.
4. Verify username and password.
5. Redirect to Secure page after successful login.
6. Use [Authorize] to protect the Secure page.
7. Logout and redirect to Login.

Code:

SOM T023

```
using System.Collections.Generic;
using System.Security.Claims;
using System.Threading.Tasks;
using System.Web;
using System.Web.UI.WebControls;
using System.Xml.Linq;
using AuthDemo.Controllers;

Program.cs
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

```
39 HomeController.cs
40 using Microsoft.AspNetCore.Mvc;
41 using Microsoft.AspNetCore.Authorization;
42 using Microsoft.AspNetCore.Authentication;
43 using Microsoft.AspNetCore.Authentication.Cookies;
44 using System.Security.Claims;
45
46 namespace AuthDemo.Controllers
47 {
48     1 reference
49     public class HomeController : Controller
50     {
51         0 references
52         public IActionResult Index()
53         {
54             return View();
55         }
56
57         [HttpPost]
58         0 references
59         public async Task<IActionResult> Index(string username, string password)
60         {
61             if (username == "admin" && password == "123")
62             {
63                 var claims = new List<Claim>
64                 {
65                     new Claim(ClaimTypes.Name, username)
66                 };
67
68                 var identity = new ClaimsIdentity(claims,
69                     CookieAuthenticationDefaults.AuthenticationScheme);
70
71                 await HttpContext.SignInAsync(
72                     CookieAuthenticationDefaults.AuthenticationScheme,
73                     new ClaimsPrincipal(identity));
74
75                 return RedirectToAction("Secure");
76             }
77
78             ViewBag.Message = "Invalid Username or Password";
79             return View();
80         }
81     }
82 }
```

```
[Authorize]
0 references
public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}

0 references
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}

Views/Home/Index.cshtml
<h2>Login</h2>

<form method="post">
    Username :
    < input type = "text" name = "username" />< br />< br />
    Password :
    < input type = "password" name = "password" />< br />< br />
    < input type = "submit" value = "Login" />
</ form >

< p style = "color:red" >
    @ViewBag.Message
</ p >
```

Output:

practical5d Home Privacy

Login

Username :

Password :

```
Welcome Som! Login Successful.
```